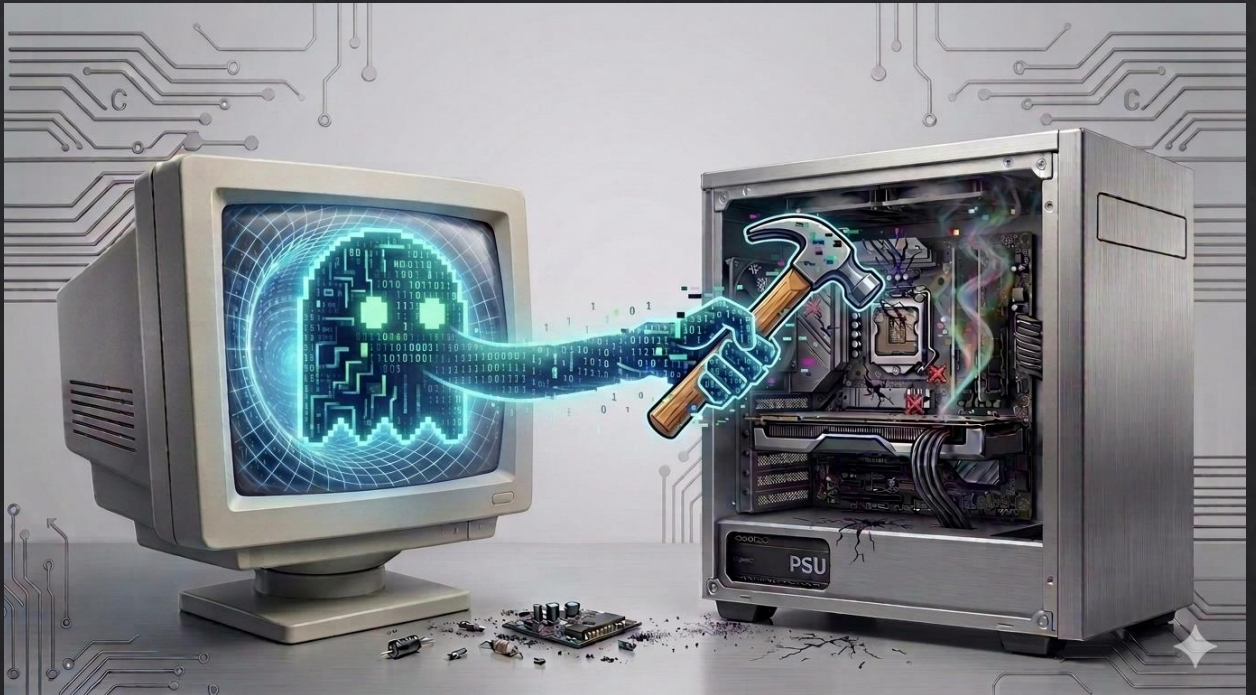


## 4. Software-based Hardware Attacks



# What you need today

---

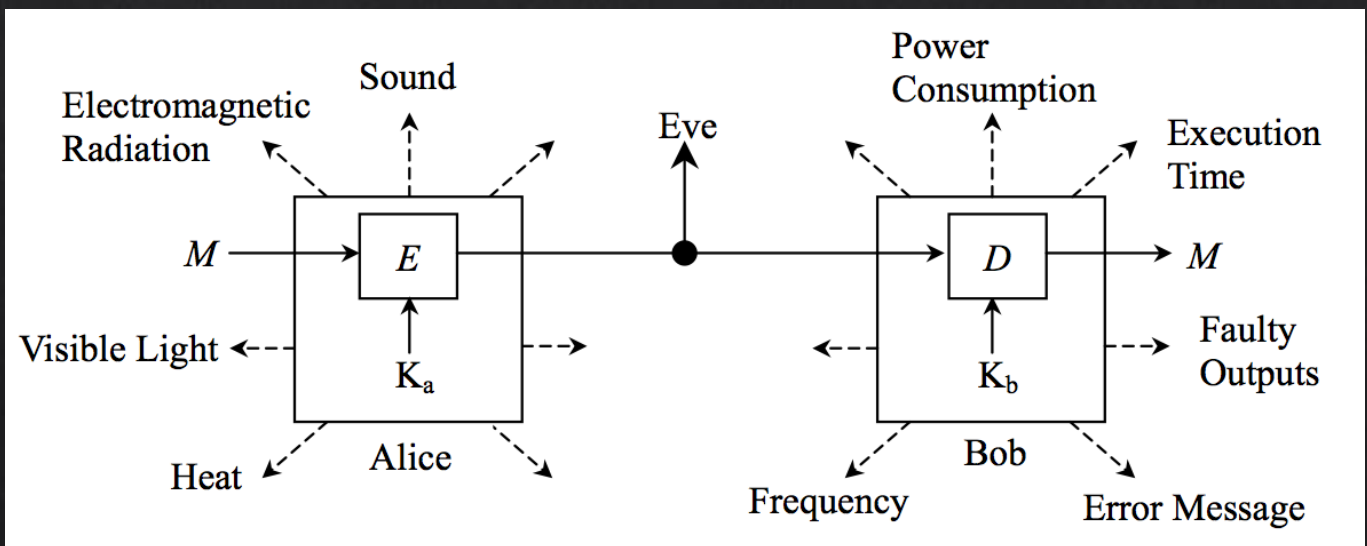
- ◆ Kali VM

- ◆ Things we do today

- ◆ Time-based side channel attack
  - ◆ Rowhammer simulation

# What is a side channel?

- ◆ Attacks based on “implementation” of a system
- ◆ Not brute-force or theoretical weaknesses
- ◆ Unintentional “features”

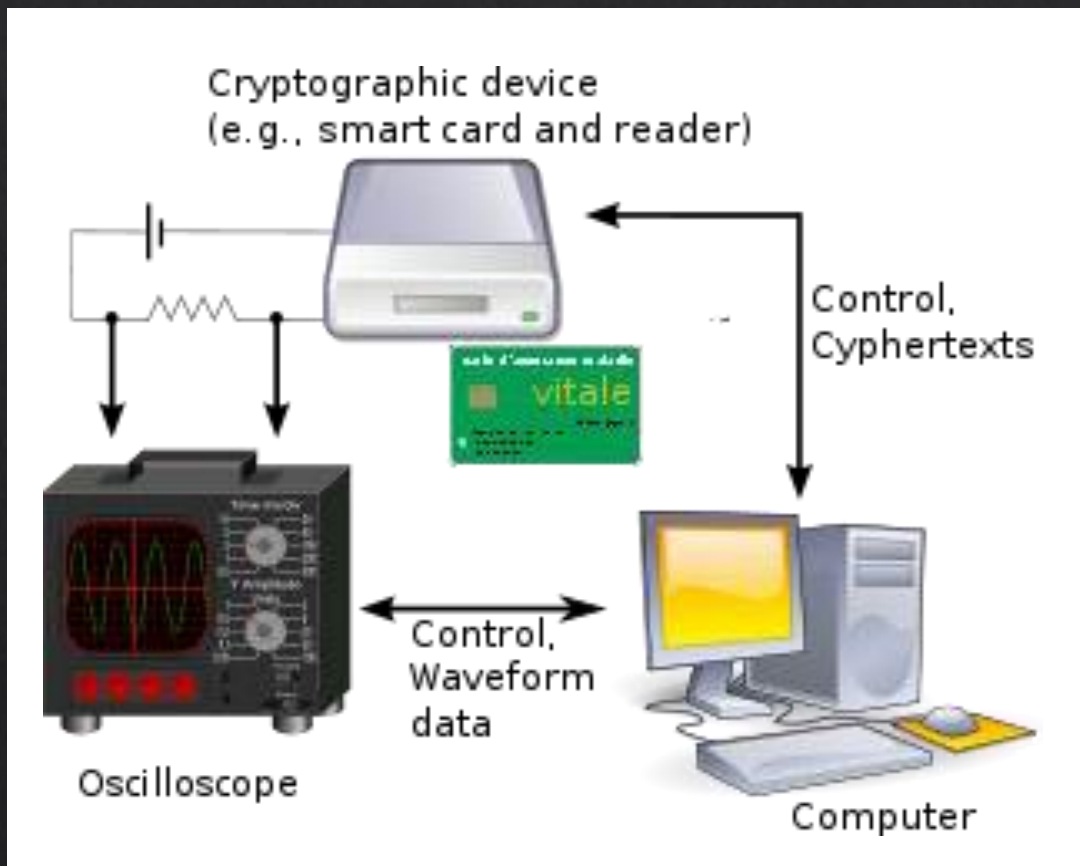


# Types of side channels

- ◆ Timing attack - how much time computations take.
- ◆ Power-monitoring attack -- varying hardware power consumption.
- ◆ Electromagnetic attacks leaked EM radiation
- ◆ Acoustic cryptanalysis: sound produced during a computation (rather like power analysis).
- ◆ Differential fault analysis: inducing faults
- ◆ Data remanence: data are read after supposedly having been deleted.
- ◆ Etc...

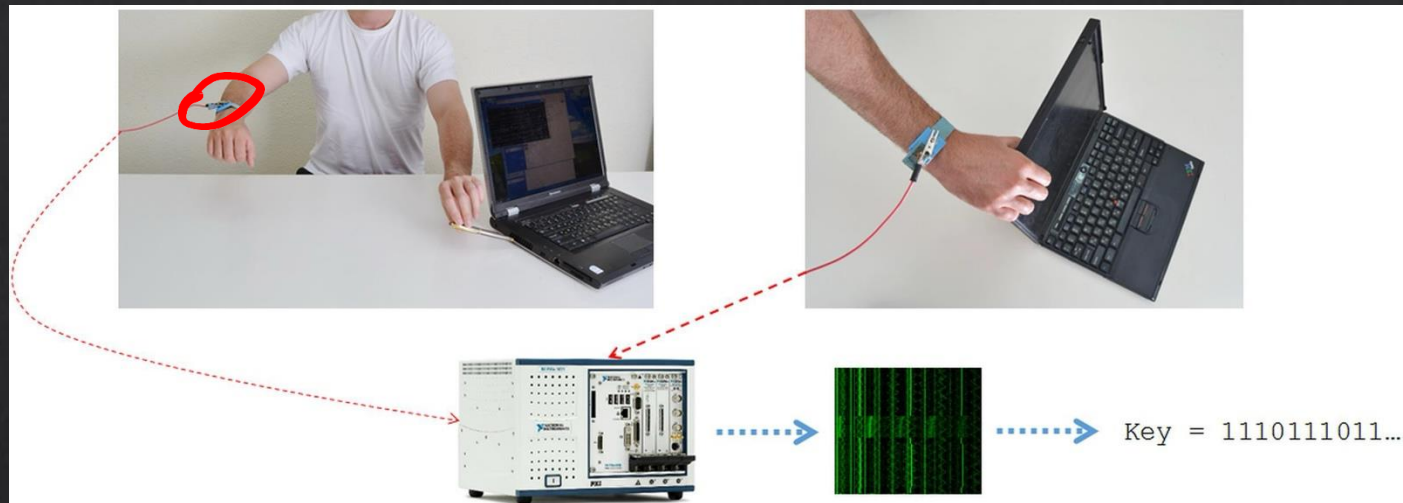
# Power Analysis

- **Simple power analysis:** Directly read off bits from powerline traces
- **Differential power analysis:** Look for statistical differences in power traces, based on guesses of a key bit

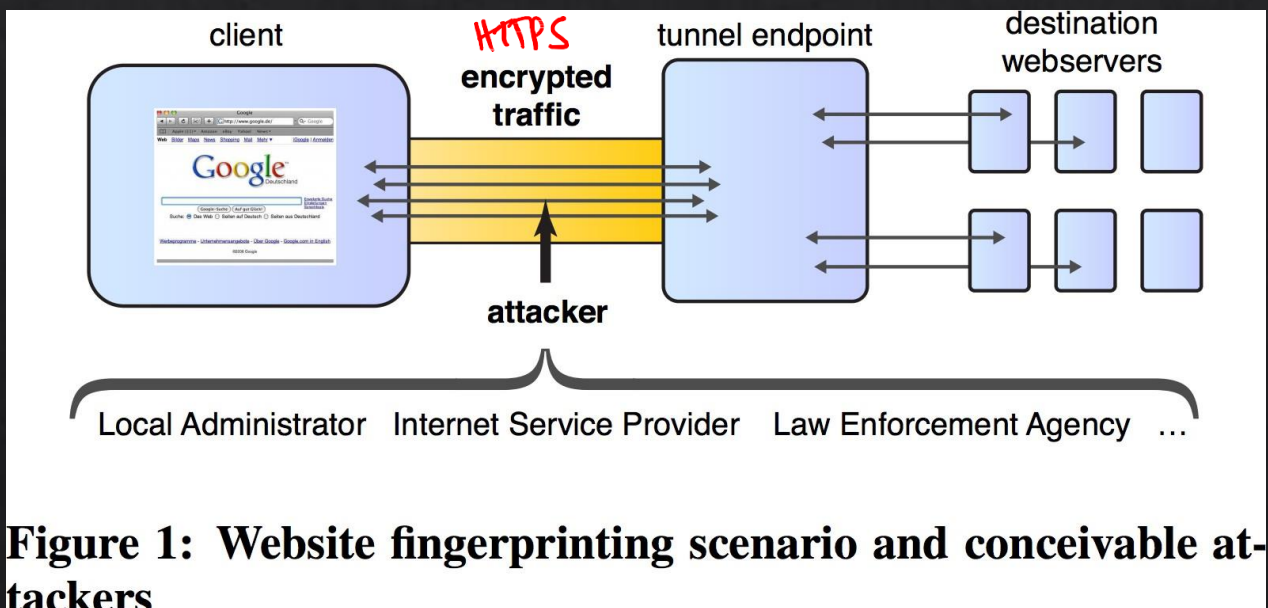


# Power Analysis Attacks

## ◆Key Extraction via Electric Potential



## ◆Identifying Web Pages: Traffic Analysis



**Figure 1: Website fingerprinting scenario and conceivable attackers**

# Timing Attacks

- ◆ RSA: Leverage key-dependent timings of modular exponentiations
  - ◆ <https://www.rambus.com/timing-attacks-on-implementations-of-diffie-hellman-rsa-dss-and-other-systems/>
  - ◆ <http://crypto.stanford.edu/~dabo/papers/ssl-timing.pdf>
- ◆ Block Ciphers: Leverage key-dependent cache hits/misses

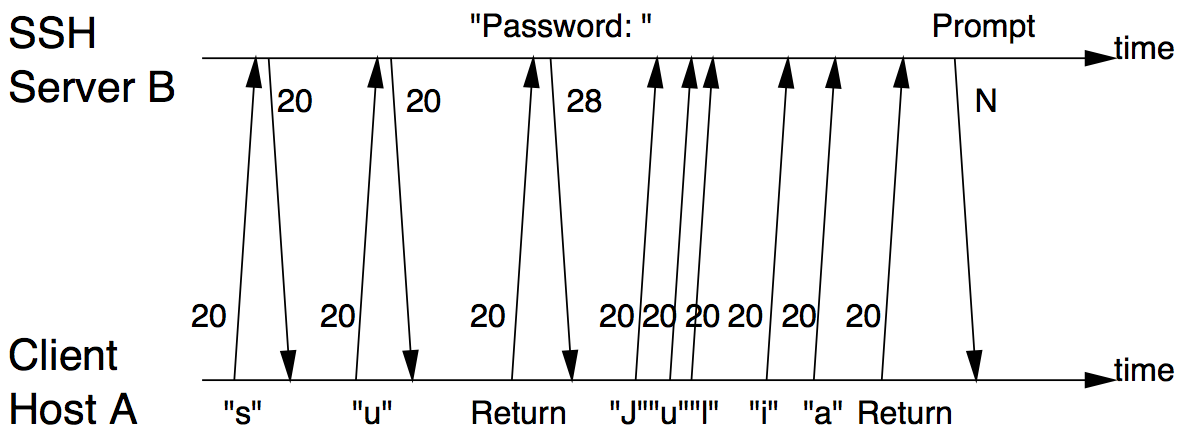
# Timing Attacks

---

- ◆SSH is widely perceived as secure alternative to telnet
- ◆Crypto is “well designed” theoretically
- ◆Can leak info such as passwords even with good theoretical design
- ◆False sense of security

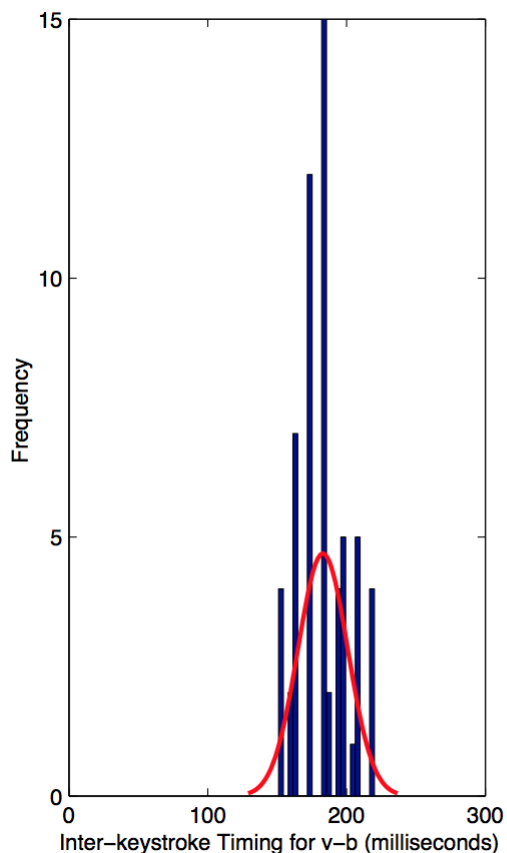
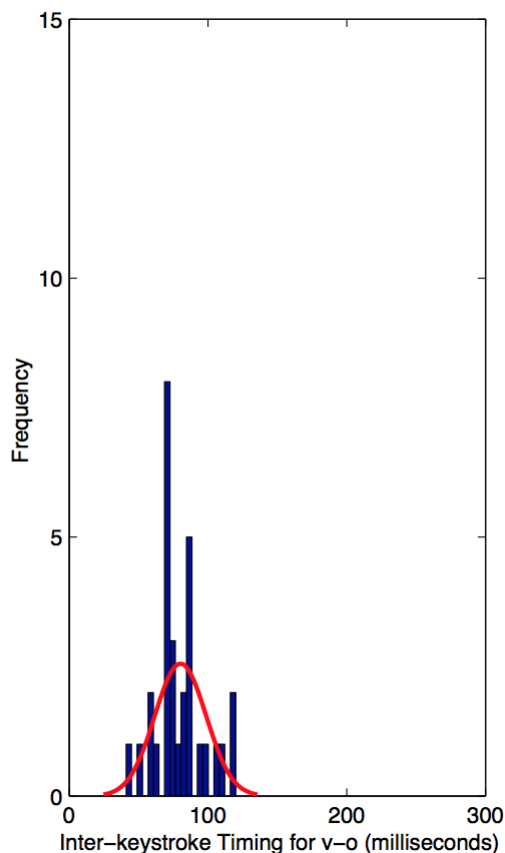
# Timing Attacks

- ◆ Packets are padded only to an eight byte boundary
  - ◆ Can learn approximate length of data
- ◆ Interactive mode optimized for low latency
  - ◆ Can learn length of passwords
  - ◆ Keystroke timings further reveal actual characters
- ◆ E.g., traffic signature for the `su` command



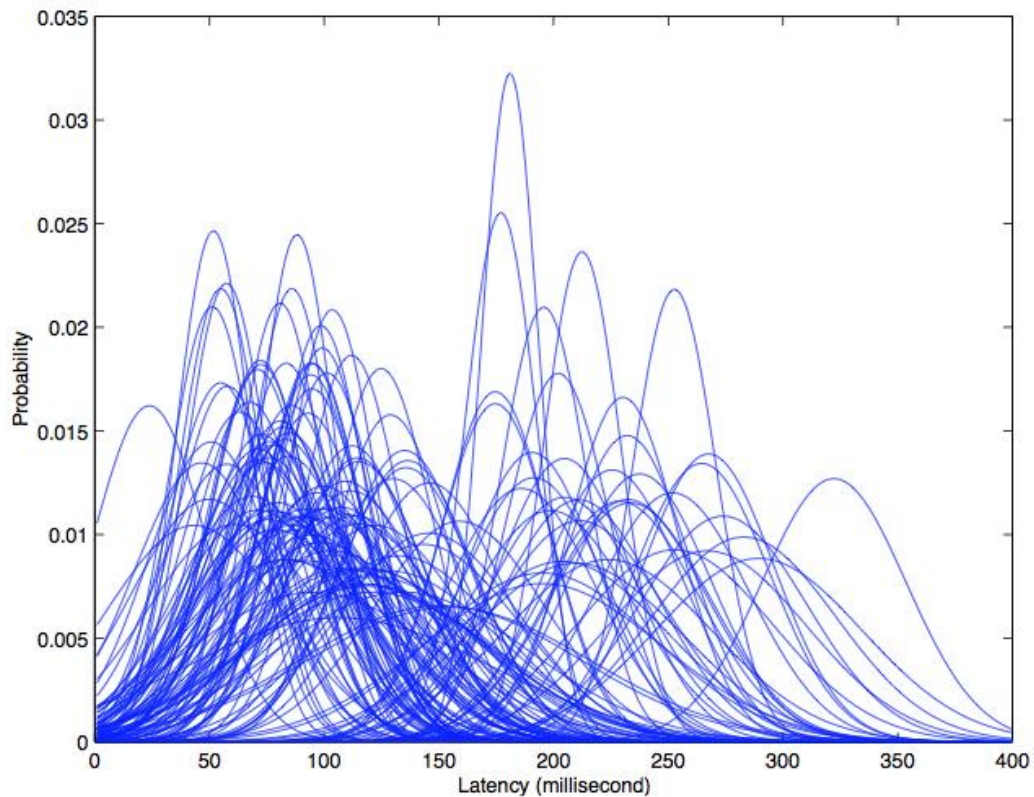
# Timing Attacks

- ◆ Focus on “touch typing”
- ◆ Most people have standard timing patterns in keystrokes, especially for passwords
- ◆ Distinguishable from inter-keystroke timings!



# Timing Attacks

Non trivial overlap  $\rightarrow$   
Inference might be hard



Gaussian models for 142 char pairs

# Timing Attacks

## ◊demo

wget [https://github.com/uwacyber/cits3006/raw/live/cits3006-labs/files/side\\_channel\\_demo.zip](https://github.com/uwacyber/cits3006/raw/live/cits3006-labs/files/side_channel_demo.zip)



# Timing Attacks

```
(base) └─(jin@kali)-[~/cits3006/lect4/side_channel_demo]  
└─$ ./login  
[login] guarding secret: "SomeRandomPassword123!@#" (24 chars)  
password: hello  
Access denied.  
password: fjdklsafsda  
Access denied.  
password: ghruieapfhew  
Access denied.  
password: SomeRandomPassword123!@#  
Access granted.  
password: ^C
```

# Timing Attacks

```
(base) └─(jin@kali)-[~/cits3006/lect4/side_channel_demo]  
└─$ ./attack ./login
```

Probing the target by timing to discover the password length (up to 8 s; Ctrl-C to skip)...

```
probing length 1/40 ... [login] guarding secret: "SomeRandomPassword123!@#" (24 chars)  
detected length: 24 (change SECRET in login.c to any length – this adapts)
```

How the length was found: for each candidate length  $L$ , the attacker sent one guess per character (that character at position 0, filler after) and timed it. At the TRUE length exactly one character runs slow – it matched a byte before the vulnerable early-exit – while every WRONG length returns instantly for every character (the ``n != secret_len`` check fails first). Length 24 was the one length with that lone slow outlier.

## TIMING SIDE-CHANNEL ATTACK – live recovery

```
Target program : ./login (a black box – we only time its replies)  
Mode           : manual
```

The attacker never sees the secret. It only sends guesses and times them.

Brute force needs up to  $74^{24}$  guesses.

Timing attack needs only  $74 \times 24 = 1776$  guesses.

# Timing Attacks

## TIMING SIDE-CHANNEL ATTACK – live recovery

Target program : `./login` (a black box – we only time its replies)  
Mode : `manual`  
The attacker never sees the secret. It only sends guesses and times them.

Brute force needs up to  $74^{24}$  guesses.  
Timing attack needs only  $74 \times 24 = 1776$  guesses.

Manual mode – commands:

|                               |                                                                 |
|-------------------------------|-----------------------------------------------------------------|
| <code>&lt;char&gt;</code>     | time one character at the current position (repeat to compare)  |
| <code>scan</code>             | sweep ALL characters at this position and rank them             |
| <code>lock &lt;c&gt;</code>   | fix character <code>c</code> here and move to the next position |
| <code>lock</code>             | fix the slowest character you've tried here, and advance        |
| <code>try &lt;text&gt;</code> | time an arbitrary full guess (freeform experiment)              |
| <code>submit</code>           | send the recovered string as-is; see granted/denied             |
| <code>detect</code>           | (re)probe the password length by timing                         |
| <code>len &lt;n&gt;</code>    | set the password length by hand (0 = unknown)                   |
| <code>auto</code>             | hand the rest over to the automatic attack                      |
| <code>help</code>             | reprint these commands                                          |
| <code>quit</code>             | exit (type the full word, or press Ctrl-D)                      |

(A single letter is always a candidate test – so you can safely try 'q'.  
On a leaky target you need the right length to see a signal: use ``detect`` or ``len``.)

```
pos 0/24 [""] > a
'a' → 2583 ns
position 0 (locked in: "") – slowest = most likely:
a                                     2583 ns
pos 0/24 [""] > b
```

# Timing Attacks

```
pos 0/24 [""] > Z
```

```
'Z' → 5333 ns
```

```
position 0 (locked in: "") – slowest = most likely:
```

|          |                                                                                    |                 |
|----------|------------------------------------------------------------------------------------|-----------------|
| <b>S</b> |  | <b>44167 ns</b> |
| g        |                                                                                    | 7083 ns         |
| Y        |                                                                                    | 5792 ns         |
| T        |                                                                                    | 5375 ns         |
| X        |                                                                                    | 5375 ns         |
| b        |                                                                                    | 5333 ns         |
| i        |                                                                                    | 5333 ns         |
| R        |                                                                                    | 5333 ns         |
| Z        |                                                                                    | 5333 ns         |
| Q        |                                                                                    | 5291 ns         |
| V        |                                                                                    | 5291 ns         |
| h        |                                                                                    | 4666 ns         |
| e        |                                                                                    | 4625 ns         |
| P        |                                                                                    | 4083 ns         |
| d        |                                                                                    | 3333 ns         |
| f        |                                                                                    | 3333 ns         |
| W        |                                                                                    | 3333 ns         |
| c        |                                                                                    | 3291 ns         |
| U        |                                                                                    | 3041 ns         |
| a        |                                                                                    | 2583 ns         |

```
pos 0/24 [""] > █
```

```
position 0 (locked in so far: "")
```

```
locked 'S' at position 0 → "S"
```

```
position 1 (locked in so far: "S")
```

|   |  |  |  |  |
|---|--|--|--|--|
| V |  |  |  |  |
| Y |  |  |  |  |
| K |  |  |  |  |
| Q |  |  |  |  |

→ no outlier here – the timings are flat. Either the target is constant-time (the fix working), or the length is wrong for a leaky target (try `detect` / `len N`).

```
locked 'o' at position 1 → "So"
```

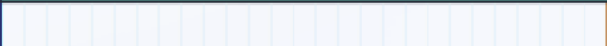
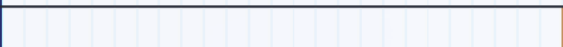
pos 2/24 [ 30 ] >

# Timing Attacks

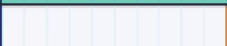
```
pos 2/24 ["So"] > auto
```

Handing over to auto...

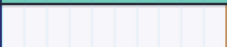
position 2 (locked in so far: "Som")

|   |                                                                                    |          |
|---|------------------------------------------------------------------------------------|----------|
| m |  | 80458 ns |
| T |   | 78750 ns |
| L |   | 78542 ns |
| E |   | 77625 ns |
| S |   | 77333 ns |
| F |   | 76875 ns |
| w |   | 76250 ns |
| * |   | 75750 ns |

position 3 (locked in so far: "Some")

|   |                                                                                      |           |
|---|--------------------------------------------------------------------------------------|-----------|
| e |  | 119833 ns |
| I |   | 105208 ns |
| J |   | 103375 ns |
| ( |   | 102750 ns |
| _ |   | 102208 ns |
| M |   | 101292 ns |
| k |   | 101041 ns |
| % |   | 100584 ns |

position 4 (locked in so far: "SomeR")

|   |                                                                                      |           |
|---|--------------------------------------------------------------------------------------|-----------|
| R |  | 149459 ns |
| I |   | 132666 ns |

# What Is RowHammer?

- ◆ One can predictably induce bit flips in commodity DRAM chips
  - ◆ >80% of the tested DRAM chips are vulnerable
  - ◆ First example of how a simple hardware failure mechanism can create a widespread system security vulnerability

WIRED

Forget Software—Now Hackers Are Exploiting Physics

BUSINESS

CULTURE

DESIGN

GEAR

SCIENCE

ANDY GREENBERG SECURITY 08.31.16 7:00 AM

SHARE



SHARE  
18276

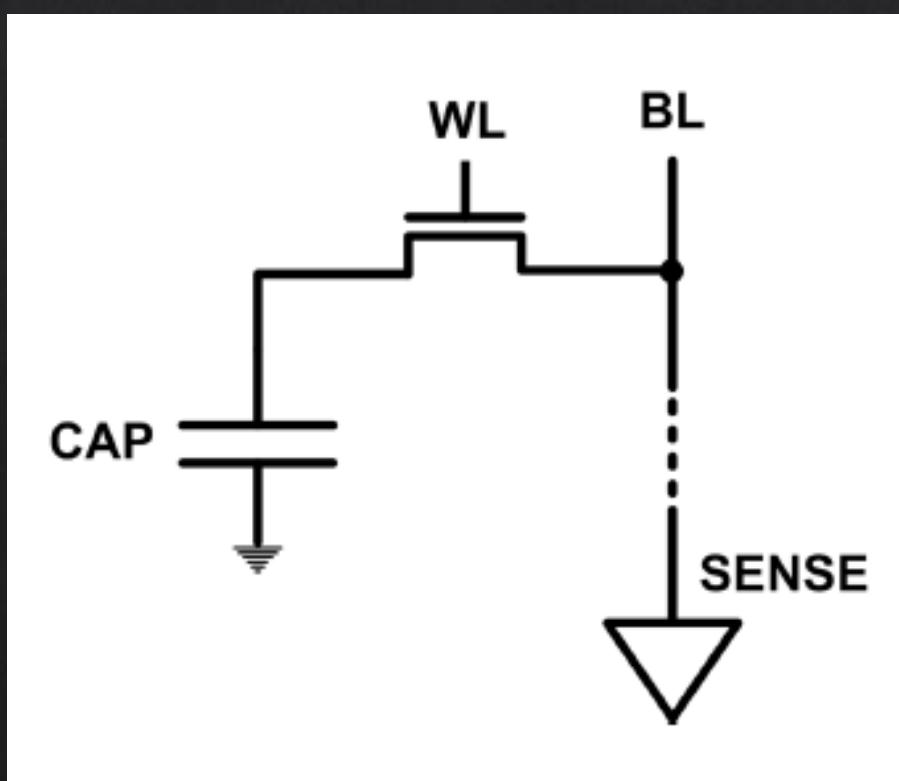


TWEET

## FORGET SOFTWARE—NOW HACKERS ARE EXPLOITING PHYSICS

# The DRAM Scaling Problem

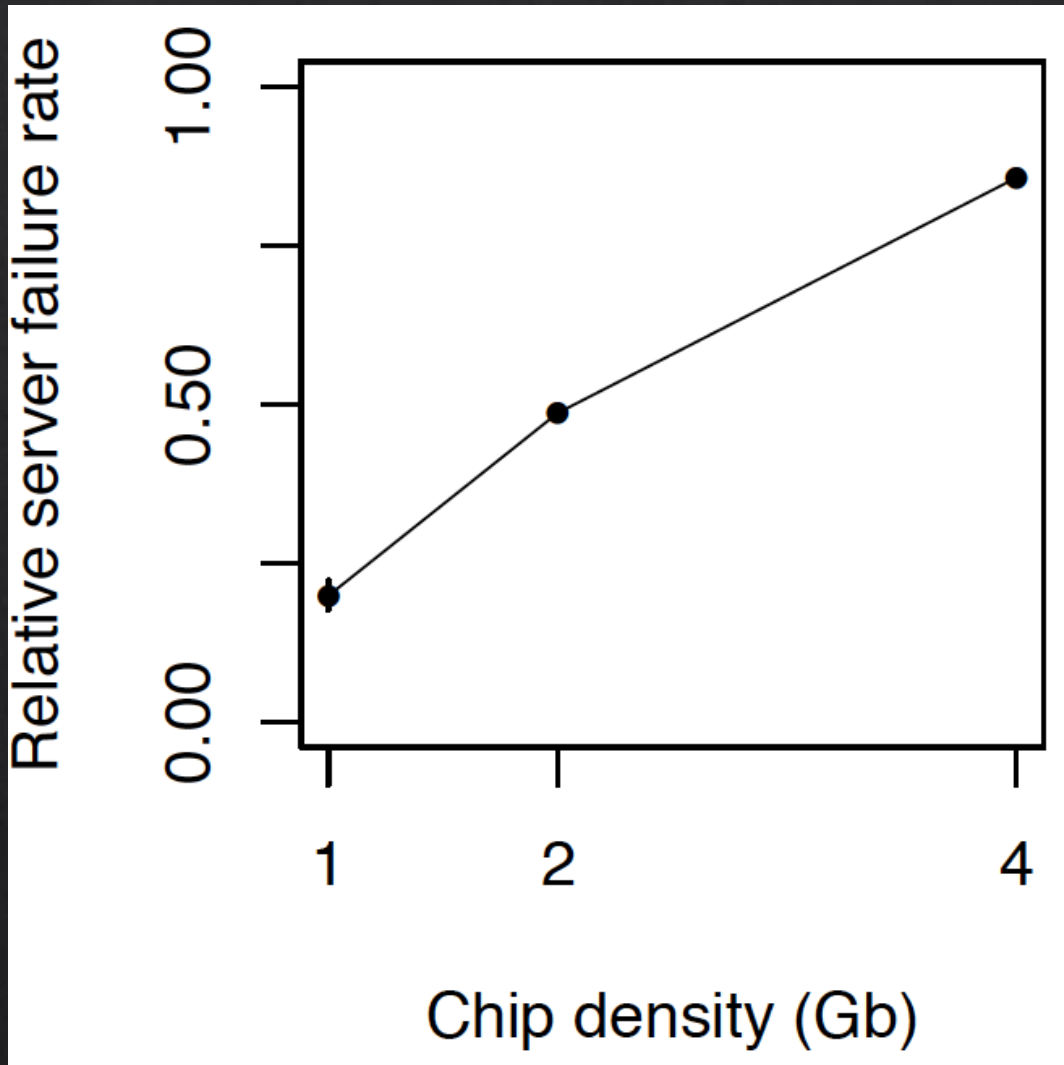
- ◆ DRAM stores charge in a capacitor (charge-based memory)
  - ◆ Capacitor must be large enough for reliable sensing
  - ◆ Access transistor should be large enough for low leakage and high retention time
  - ◆ Scaling beyond 40-35nm (2013) is challenging [ITRS, 2009]
- ◆ DRAM capacity, cost, and energy/power hard to scale



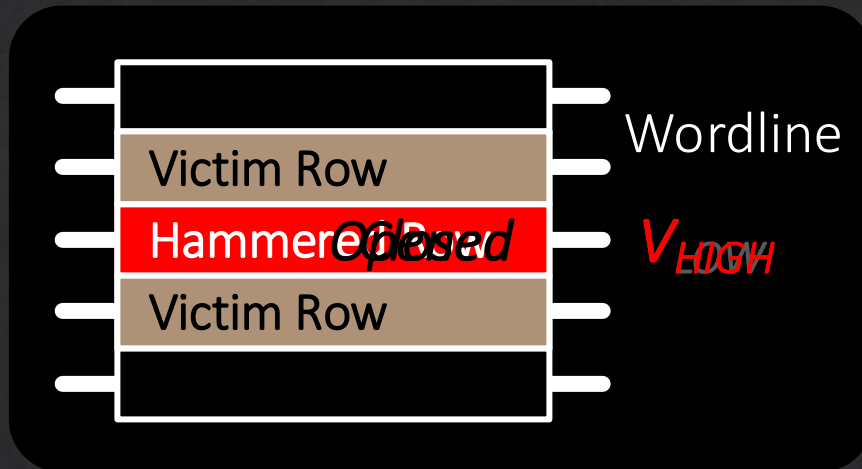
# As Memory Scales, It Becomes Unreliable

◆ Data from all of Facebook's servers worldwide

◆ Meza+, "Revisiting Memory Errors in Large-Scale Production Data Centers," DSN'15.



# Modern DRAM is Prone to Disturbance Errors



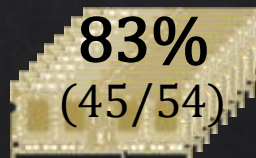
Repeatedly reading a row enough times (before memory gets refreshed) induces **disturbance errors** in adjacent rows in **most real DRAM chips you can buy today**

**A** company



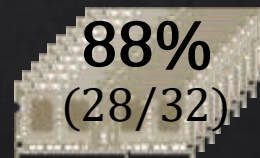
Up to  
 $1.0 \times 10^7$   
errors

**B** company



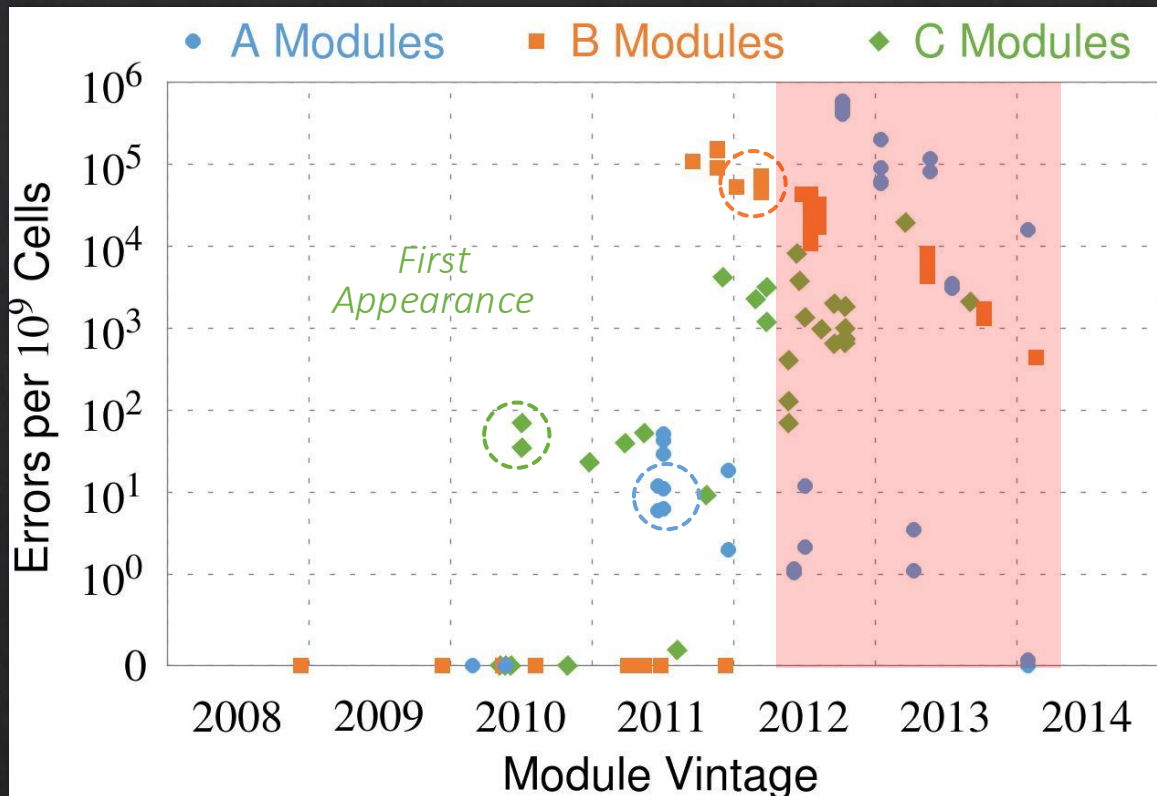
Up to  
 $2.7 \times 10^6$   
errors

**C** company



Up to  
 $3.3 \times 10^5$   
errors

# Recent DRAM Is More Vulnerable



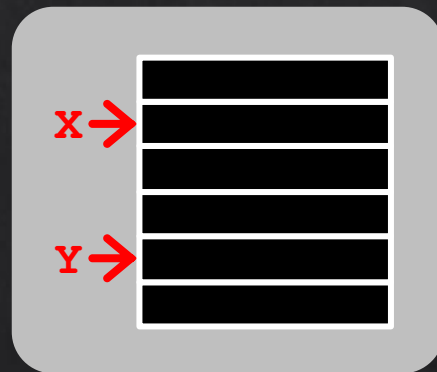
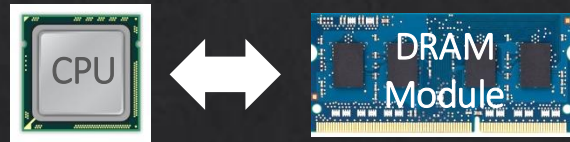
*All modules from 2012-2013 are vulnerable*

# Why Is This Happening?

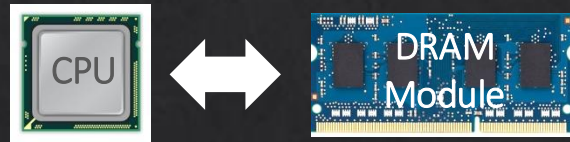
- ◆ DRAM cells are too close to each other!
  - ◆ They are not electrically isolated from each other
- ◆ Access to one cell affects the value in nearby cells
  - ◆ due to electrical interference between
    - ◆ the cells
    - ◆ wires used for accessing the cells
  - ◆ Also called cell-to-cell coupling/interference
- ◆ Example: When we activate (apply high voltage) to a row, an adjacent row gets slightly activated as well
  - ◆ Vulnerable cells in that slightly-activated row lose a little bit of charge
  - ◆ If RowHammer happens enough times, charge in such cells gets drained

# A Simple Program Can Induce Many Errors

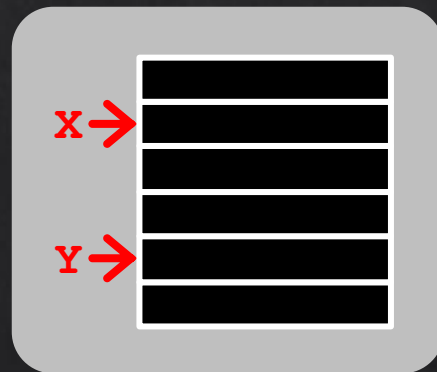
```
loop:  
  mov (X), %eax  
  mov (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp loop
```



# A Simple Program Can Induce Many Errors

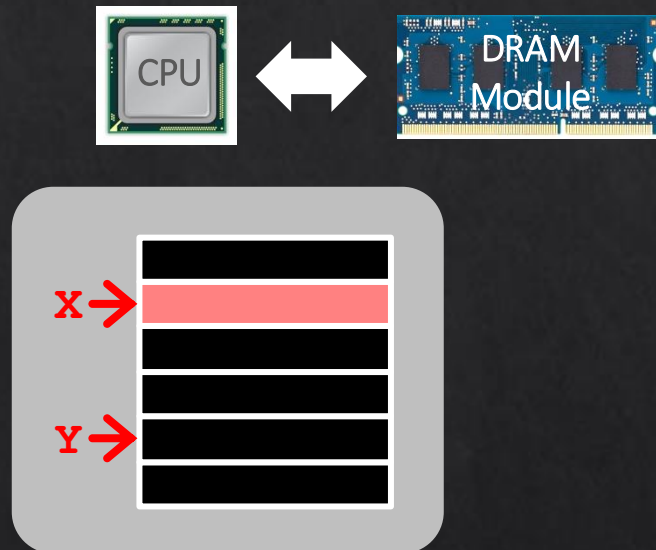


1. Avoid *cache hits*
  - Flush **x** from cache
2. Avoid *row hits* to **x**
  - Read **y** in another row



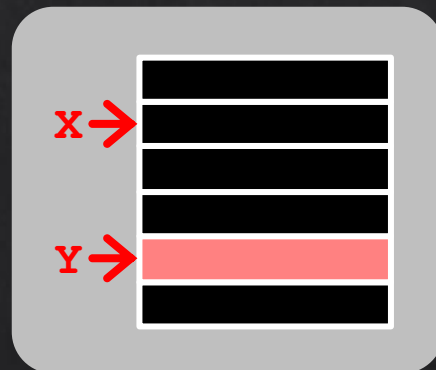
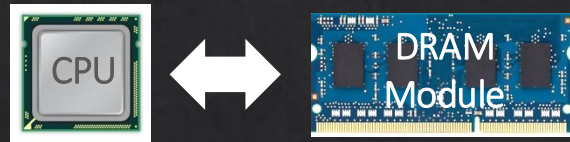
# A Simple Program Can Induce Many Errors

```
loop:  
  mov (X), %eax  
  mov (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp loop
```



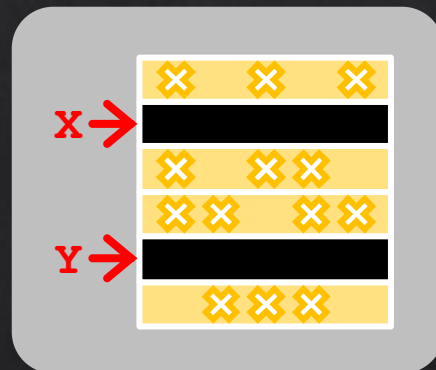
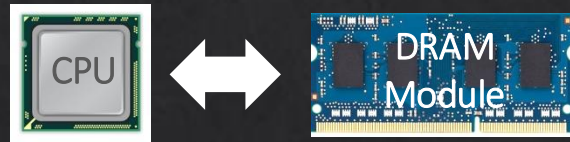
# A Simple Program Can Induce Many Errors

```
loop:  
  mov (X), %eax  
  mov (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp loop
```



# A Simple Program Can Induce Many Errors

```
loop:  
  mov (X), %eax  
  mov (Y), %ebx  
  clflush (X)  
  clflush (Y)  
  mfence  
  jmp loop
```



# Observed Errors in Real Systems

| CPU Architecture          | Errors | Access-Rate |
|---------------------------|--------|-------------|
| Intel Haswell (2013)      | 22.9K  | 12.3M/sec   |
| Intel Ivy Bridge (2012)   | 20.7K  | 11.7M/sec   |
| Intel Sandy Bridge (2011) | 16.1K  | 11.6M/sec   |
| AMD Piledriver (2012)     | 59     | 6.1M/sec    |

# Security Implications



It's like breaking into an apartment by repeatedly slamming a neighbor's door until the vibrations open the door you were after

# More Security Implications

**"We can gain unrestricted access to systems of website visitors."**

Not there yet, but ...



ROOT privileges for web apps!

29 Daniel Gruss (@lavados), Clémentine Maurice (@BloodyTangerine),  
December 28, 2015 — 32c3, Hamburg, Germany

www.isik.tugraz.at



GATED COMMUNITIES

Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript (DIMVA'16)

**"Can gain control of a smart phone deterministically"**

*Hammer And Root*



ANDROID

*Millions of Androids*

Drammer: Deterministic Rowhammer Attacks on Mobile Platforms, CCS'16

# More Security Implications

- ◆ Using an integrated GPU in a mobile system to remotely escalate privilege via the WebGL interface - IEEE S&P 2018



BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE

"GRAND PWINING UNIT" —

## Drive-by Rowhammer attack uses GPU to compromise an Android phone

JavaScript based GLitch pwns browsers by flipping bits inside memory chips.

DAN GOODIN - 5/3/2018, 12:00 PM

## Grand Pwning Unit: Accelerating Microarchitectural Attacks with the GPU

Pietro Frigo  
Vrije Universiteit  
Amsterdam  
p.frigo@vu.nl

Cristiano Giuffrida  
Vrije Universiteit  
Amsterdam  
giuffrida@cs.vu.nl

Herbert Bos  
Vrije Universiteit  
Amsterdam  
herbertb@cs.vu.nl

Kaveh Razavi  
Vrije Universiteit  
Amsterdam  
kaveh@cs.vu.nl

# More Security Implications

## ◆ Rowhammer over RDMA (I) - USENIX ATC 2018

 ars TECHNICA

BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE

THROWHAMMER —

### Packets over a LAN are all it takes to trigger serious Rowhammer bit flips

The bar for exploiting potentially serious DDR weakness keeps getting lower.

DAN GOODIN - 5/10/2018, 5:26 PM

### Throwhammer: Rowhammer Attacks over the Network and Defenses

Andrei Tatar  
*VU Amsterdam*

Radhesh Krishnan  
*VU Amsterdam*

Elias Athanasopoulos  
*University of Cyprus*

Cristiano Giuffrida  
*VU Amsterdam*

Herbert Bos  
*VU Amsterdam*

Kaveh Razavi  
*VU Amsterdam*



### Nethammer—Exploiting DRAM Rowhammer Bug Through Network Requests

### Nethammer: Inducing Rowhammer Faults through Network Requests

Moritz Lipp  
Graz University of Technology

Misiker Tadesse Aga  
University of Michigan

Michael Schwarz  
Graz University of Technology

Daniel Gruss  
Graz University of Technology

Clémentine Maurice  
Univ Rennes, CNRS, IRISA

Lukas Raab  
Graz University of Technology

Lukas Lamster  
Graz University of Technology

# More Security Implications

◆ IEEE S&P 2020

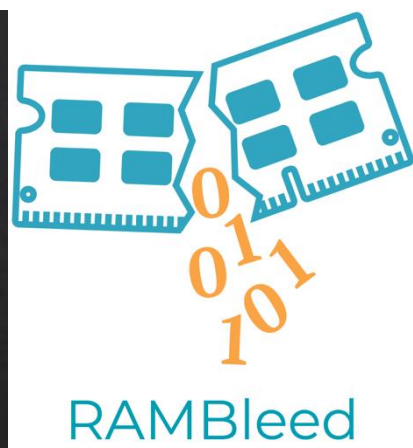
## RAMBleed: Reading Bits in Memory Without Accessing Them

Andrew Kwong  
University of Michigan  
ankwong@umich.edu

Daniel Genkin  
University of Michigan  
genkin@umich.edu

Daniel Gruss  
Graz University of Technology  
daniel.gruss@iaik.tugraz.at

Yuval Yarom  
University of Adelaide and Data61  
yval@cs.adelaide.edu.au



◆ USENIX Security 2019



### A Single Bit-flip Can Cause Terminal Brain Damage to DNNs

*One specific bit-flip in a DNN's representation leads to accuracy drop over 90%*

Our research found that a specific bit-flip in a DNN's bitwise representation can cause the accuracy loss up to 90%, and the DNN has 40-50% parameters, on average, that can lead to the accuracy drop over 10% when individually subjected to such single bitwise corruptions...

[Read More](#)

### Terminal Brain Damage: Exposing the Graceless Degradation in Deep Neural Networks Under Hardware Fault Attacks

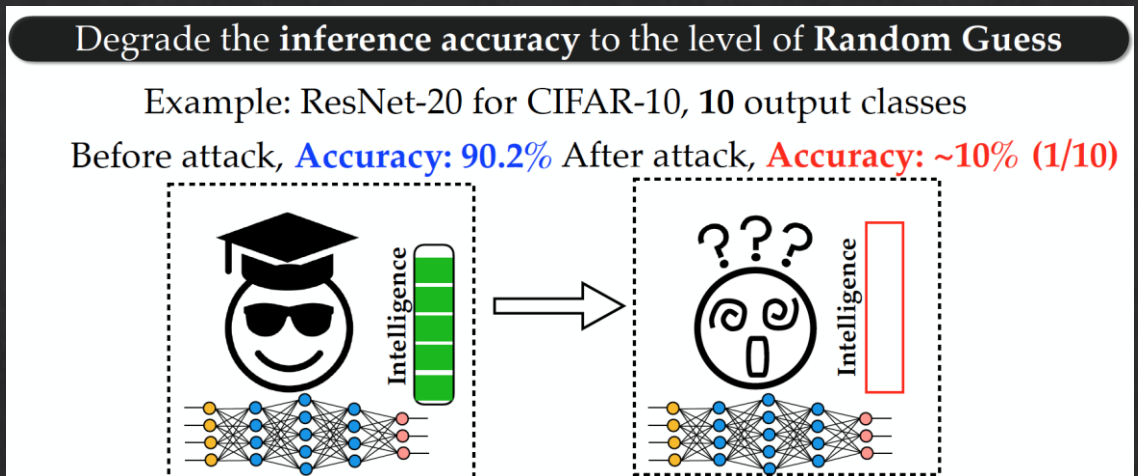
Sanghyun Hong, Pietro Frigo<sup>†</sup>, Yiğitcan Kaya, Cristiano Giuffrida<sup>†</sup>, Tudor Dumitras

University of Maryland, College Park

<sup>†</sup>Vrije Universiteit Amsterdam

# More Security Implications (VIII)

## ◆USENIX Security 2020



## DeepHammer: Depleting the Intelligence of Deep Neural Networks through Targeted Chain of Bit Flips

Fan Yao  
University of Central Florida  
[fan.yao@ucf.edu](mailto:fan.yao@ucf.edu)

Adnan Siraj Rakin  
Arizona State University  
[asrakin@asu.edu](mailto:asrakin@asu.edu)

Deliang Fan  
Arizona State University  
[dfan@asu.edu](mailto:dfan@asu.edu)

# More Security Implications (IX)

- ◆ Rowhammer on MLC NAND Flash (based on [Cai+, HPCA 2017])

**The Register**<sup>®</sup>

*Biting the hand that feeds IT*

Security

## Rowhammer RAM attack adapted to hit flash storage

Project Zero's two-year-old dog learns a new trick

By [Richard Chirgwin](#) 17 Aug 2017 at 04:27

17  SHARE ▼

### **From random block corruption to privilege escalation: A filesystem attack vector for rowhammer-like attacks**

Anil Kurmus

Nikolas Ioannou

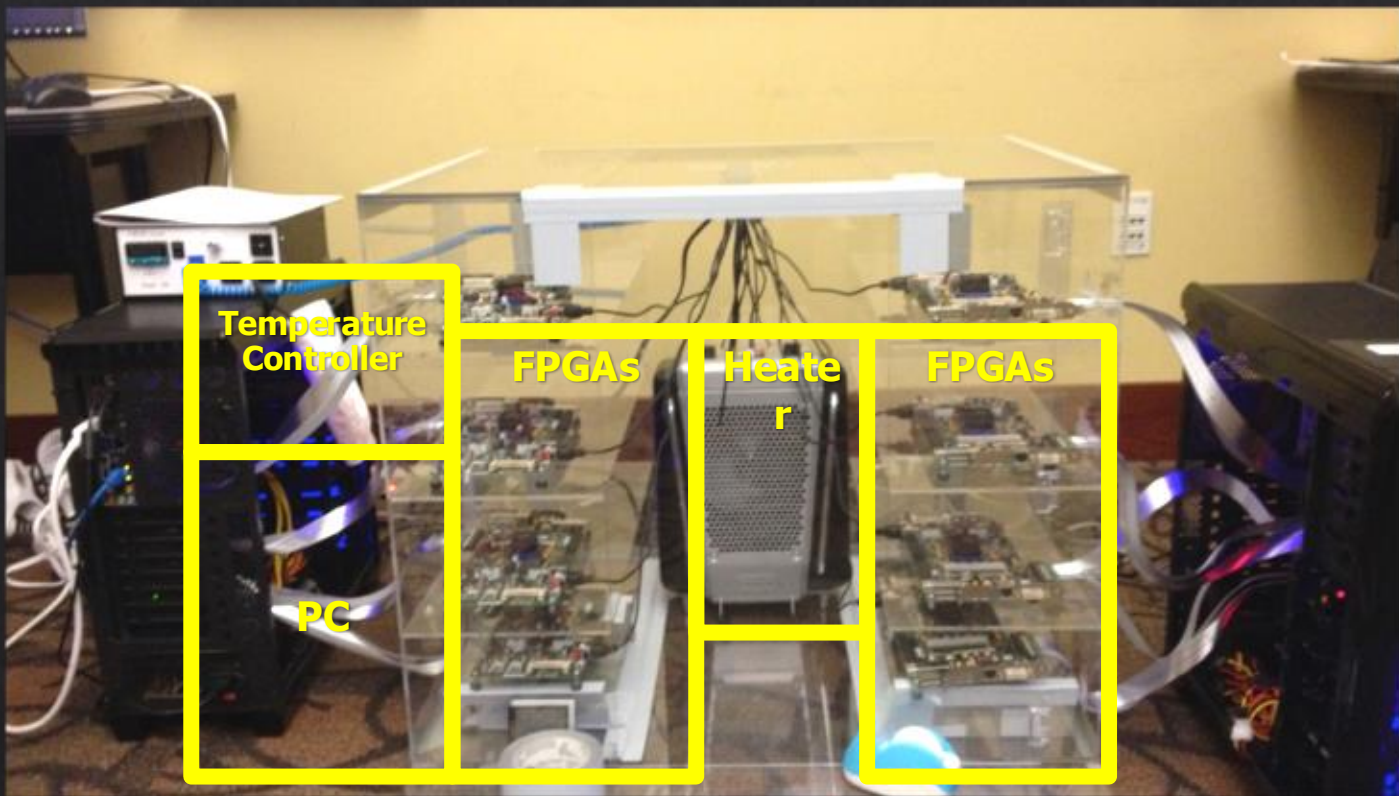
Matthias Neugschwandtner

Nikolaos Papandreou

Thomas Parnell

*IBM Research – Zurich*

# RowHammer Experiment



| Manufacturer        | Module             | Date*   | Timing <sup>†</sup> |                      | Organization |       | Chip                   |      |                         | Victims-per-Module    |                       |                       | RI <sub>th</sub> (ms) |
|---------------------|--------------------|---------|---------------------|----------------------|--------------|-------|------------------------|------|-------------------------|-----------------------|-----------------------|-----------------------|-----------------------|
|                     |                    | (yy-ww) | Freq (MT/s)         | t <sub>RC</sub> (ns) | Size (GB)    | Chips | Size (Gb) <sup>‡</sup> | Pins | DieVersion <sup>§</sup> | Average               | Minimum               | Maximum               | Min                   |
| A                   | A <sub>1</sub>     | 10-08   | 1066                | 50.625               | 0.5          | 4     | 1                      | ×16  | B                       | 0                     | 0                     | 0                     | –                     |
|                     | A <sub>2</sub>     | 10-20   | 1066                | 50.625               | 1            | 8     | 1                      | ×8   | F                       | 0                     | 0                     | 0                     | –                     |
|                     | A <sub>3-5</sub>   | 10-20   | 1066                | 50.625               | 0.5          | 4     | 1                      | ×16  | B                       | 0                     | 0                     | 0                     | –                     |
|                     | A <sub>6-7</sub>   | 11-24   | 1066                | 49.125               | 1            | 4     | 2                      | ×16  | D                       | 7.8 × 10 <sup>1</sup> | 5.2 × 10 <sup>1</sup> | 1.0 × 10 <sup>2</sup> | 21.3                  |
|                     | A <sub>8-12</sub>  | 11-26   | 1066                | 49.125               | 1            | 4     | 2                      | ×16  | D                       | 2.4 × 10 <sup>2</sup> | 5.4 × 10 <sup>1</sup> | 4.4 × 10 <sup>2</sup> | 16.4                  |
|                     | A <sub>13-14</sub> | 11-50   | 1066                | 49.125               | 1            | 4     | 2                      | ×16  | D                       | 8.8 × 10 <sup>1</sup> | 1.7 × 10 <sup>1</sup> | 1.6 × 10 <sup>2</sup> | 26.2                  |
|                     | A <sub>15-16</sub> | 12-22   | 1600                | 50.625               | 1            | 4     | 2                      | ×16  | D                       | 9.5                   | 9                     | 1.0 × 10 <sup>1</sup> | 34.4                  |
|                     | A <sub>17-18</sub> | 12-26   | 1600                | 49.125               | 2            | 8     | 2                      | ×8   | M                       | 1.2 × 10 <sup>2</sup> | 3.7 × 10 <sup>1</sup> | 2.0 × 10 <sup>2</sup> | 21.3                  |
|                     | A <sub>19-30</sub> | 12-40   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | K                       | 8.6 × 10 <sup>6</sup> | 7.0 × 10 <sup>6</sup> | 1.0 × 10 <sup>7</sup> | 8.2                   |
|                     | A <sub>31-34</sub> | 13-02   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | –                       | 1.8 × 10 <sup>6</sup> | 1.0 × 10 <sup>6</sup> | 3.5 × 10 <sup>6</sup> | 11.5                  |
|                     | A <sub>35-36</sub> | 13-14   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | –                       | 4.0 × 10 <sup>1</sup> | 1.9 × 10 <sup>1</sup> | 6.1 × 10 <sup>1</sup> | 21.3                  |
|                     | A <sub>37-38</sub> | 13-20   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | K                       | 1.7 × 10 <sup>6</sup> | 1.4 × 10 <sup>6</sup> | 2.0 × 10 <sup>6</sup> | 9.8                   |
|                     | A <sub>39-40</sub> | 13-28   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | K                       | 5.7 × 10 <sup>4</sup> | 5.4 × 10 <sup>4</sup> | 6.0 × 10 <sup>4</sup> | 16.4                  |
| B                   | A <sub>41</sub>    | 14-04   | 1600                | 49.125               | 2            | 8     | 2                      | ×8   | –                       | 2.7 × 10 <sup>5</sup> | 2.7 × 10 <sup>5</sup> | 2.7 × 10 <sup>5</sup> | 18.0                  |
|                     | A <sub>42-43</sub> | 14-04   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | K                       | 0.5                   | 0                     | 1                     | 62.3                  |
|                     | B <sub>1</sub>     | 08-49   | 1066                | 50.625               | 1            | 8     | 1                      | ×8   | D                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>2</sub>     | 09-49   | 1066                | 50.625               | 1            | 8     | 1                      | ×8   | E                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>3</sub>     | 10-19   | 1066                | 50.625               | 1            | 8     | 1                      | ×8   | F                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>4</sub>     | 10-31   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | C                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>5</sub>     | 11-13   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | C                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>6</sub>     | 11-16   | 1066                | 50.625               | 1            | 8     | 1                      | ×8   | F                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>7</sub>     | 11-19   | 1066                | 50.625               | 1            | 8     | 1                      | ×8   | F                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>8</sub>     | 11-25   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | C                       | 0                     | 0                     | 0                     | –                     |
|                     | B <sub>9</sub>     | 11-37   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | D                       | 1.9 × 10 <sup>6</sup> | 1.9 × 10 <sup>6</sup> | 1.9 × 10 <sup>6</sup> | 11.5                  |
|                     | B <sub>10-12</sub> | 11-46   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | D                       | 2.2 × 10 <sup>6</sup> | 1.5 × 10 <sup>6</sup> | 2.7 × 10 <sup>6</sup> | 11.5                  |
|                     | B <sub>13</sub>    | 11-49   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | C                       | 0                     | 0                     | 0                     | –                     |
| C                   | B <sub>14</sub>    | 12-01   | 1866                | 47.125               | 2            | 8     | 2                      | ×8   | D                       | 9.1 × 10 <sup>5</sup> | 9.1 × 10 <sup>5</sup> | 9.1 × 10 <sup>5</sup> | 9.8                   |
|                     | B <sub>15-31</sub> | 12-10   | 1866                | 47.125               | 2            | 8     | 2                      | ×8   | D                       | 9.8 × 10 <sup>5</sup> | 7.8 × 10 <sup>5</sup> | 1.2 × 10 <sup>6</sup> | 11.5                  |
|                     | B <sub>32</sub>    | 12-25   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | E                       | 7.4 × 10 <sup>5</sup> | 7.4 × 10 <sup>5</sup> | 7.4 × 10 <sup>5</sup> | 11.5                  |
|                     | B <sub>33-42</sub> | 12-28   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | E                       | 5.2 × 10 <sup>5</sup> | 1.9 × 10 <sup>5</sup> | 7.3 × 10 <sup>5</sup> | 11.5                  |
|                     | B <sub>43-47</sub> | 12-31   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | E                       | 4.0 × 10 <sup>5</sup> | 2.9 × 10 <sup>5</sup> | 5.5 × 10 <sup>5</sup> | 13.1                  |
|                     | B <sub>48-51</sub> | 13-19   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | E                       | 1.1 × 10 <sup>5</sup> | 7.4 × 10 <sup>4</sup> | 1.4 × 10 <sup>5</sup> | 14.7                  |
|                     | B <sub>52-53</sub> | 13-40   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | D                       | 2.6 × 10 <sup>4</sup> | 2.3 × 10 <sup>4</sup> | 2.9 × 10 <sup>4</sup> | 21.3                  |
|                     | B <sub>54</sub>    | 14-07   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | D                       | 7.5 × 10 <sup>3</sup> | 7.5 × 10 <sup>3</sup> | 7.5 × 10 <sup>3</sup> | 26.2                  |
|                     | C <sub>1</sub>     | 10-18   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | A                       | 0                     | 0                     | 0                     | –                     |
|                     | C <sub>2</sub>     | 10-20   | 1066                | 50.625               | 2            | 8     | 2                      | ×8   | A                       | 0                     | 0                     | 0                     | –                     |
|                     | C <sub>3</sub>     | 10-22   | 1066                | 50.625               | 2            | 8     | 2                      | ×8   | A                       | 0                     | 0                     | 0                     | –                     |
|                     | C <sub>4-5</sub>   | 10-26   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | B                       | 8.9 × 10 <sup>2</sup> | 6.0 × 10 <sup>2</sup> | 1.2 × 10 <sup>3</sup> | 29.5                  |
|                     | C <sub>6</sub>     | 10-43   | 1333                | 49.125               | 1            | 8     | 1                      | ×8   | T                       | 0                     | 0                     | 0                     | –                     |
|                     | C <sub>7</sub>     | 10-51   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | B                       | 4.0 × 10 <sup>2</sup> | 4.0 × 10 <sup>2</sup> | 4.0 × 10 <sup>2</sup> | 29.5                  |
|                     | C <sub>8</sub>     | 11-12   | 1333                | 46.25                | 2            | 8     | 2                      | ×8   | B                       | 6.9 × 10 <sup>2</sup> | 6.9 × 10 <sup>2</sup> | 6.9 × 10 <sup>2</sup> | 21.3                  |
|                     | C <sub>9</sub>     | 11-19   | 1333                | 46.25                | 2            | 8     | 2                      | ×8   | B                       | 9.2 × 10 <sup>2</sup> | 9.2 × 10 <sup>2</sup> | 9.2 × 10 <sup>2</sup> | 27.9                  |
|                     | C <sub>10</sub>    | 11-31   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | B                       | 3                     | 3                     | 3                     | 39.3                  |
|                     | C <sub>11</sub>    | 11-42   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | B                       | 1.6 × 10 <sup>2</sup> | 1.6 × 10 <sup>2</sup> | 1.6 × 10 <sup>2</sup> | 39.3                  |
|                     | C <sub>12</sub>    | 11-48   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 7.1 × 10 <sup>4</sup> | 7.1 × 10 <sup>4</sup> | 7.1 × 10 <sup>4</sup> | 19.7                  |
| Total of 32 Modules | C <sub>13</sub>    | 12-08   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | C                       | 3.9 × 10 <sup>4</sup> | 3.9 × 10 <sup>4</sup> | 3.9 × 10 <sup>4</sup> | 21.3                  |
|                     | C <sub>14-15</sub> | 12-12   | 1333                | 49.125               | 2            | 8     | 2                      | ×8   | C                       | 3.7 × 10 <sup>4</sup> | 2.1 × 10 <sup>4</sup> | 5.4 × 10 <sup>4</sup> | 21.3                  |
|                     | C <sub>16-18</sub> | 12-20   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 3.5 × 10 <sup>3</sup> | 1.2 × 10 <sup>3</sup> | 7.0 × 10 <sup>3</sup> | 27.9                  |
|                     | C <sub>19</sub>    | 12-23   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | E                       | 1.4 × 10 <sup>5</sup> | 1.4 × 10 <sup>5</sup> | 1.4 × 10 <sup>5</sup> | 18.0                  |
|                     | C <sub>20</sub>    | 12-24   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 6.5 × 10 <sup>4</sup> | 6.5 × 10 <sup>4</sup> | 6.5 × 10 <sup>4</sup> | 21.3                  |
|                     | C <sub>21</sub>    | 12-26   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 2.3 × 10 <sup>4</sup> | 2.3 × 10 <sup>4</sup> | 2.3 × 10 <sup>4</sup> | 24.6                  |
|                     | C <sub>22</sub>    | 12-32   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 1.7 × 10 <sup>4</sup> | 1.7 × 10 <sup>4</sup> | 1.7 × 10 <sup>4</sup> | 22.9                  |
|                     | C <sub>23-24</sub> | 12-37   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 2.3 × 10 <sup>4</sup> | 1.1 × 10 <sup>4</sup> | 3.4 × 10 <sup>4</sup> | 18.0                  |
|                     | C <sub>25-30</sub> | 12-41   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 2.0 × 10 <sup>4</sup> | 1.1 × 10 <sup>4</sup> | 3.2 × 10 <sup>4</sup> | 19.7                  |
|                     | C <sub>31</sub>    | 13-11   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 3.3 × 10 <sup>5</sup> | 3.3 × 10 <sup>5</sup> | 3.3 × 10 <sup>5</sup> | 14.7                  |
|                     | C <sub>32</sub>    | 13-35   | 1600                | 48.125               | 2            | 8     | 2                      | ×8   | C                       | 3.7 × 10 <sup>4</sup> | 3.7 × 10 <sup>4</sup> | 3.7 × 10 <sup>4</sup> | 21.3                  |

\* We report the manufacture date marked on the chip packages, which is more accurate than other dates that can be gleaned from a module.

† We report timing constraints stored in the module's on-board ROM [33], which is read by the system BIOS to calibrate the memory controller.

‡ The maximum DRAM chip size supported by our testing platform is 2Gb.

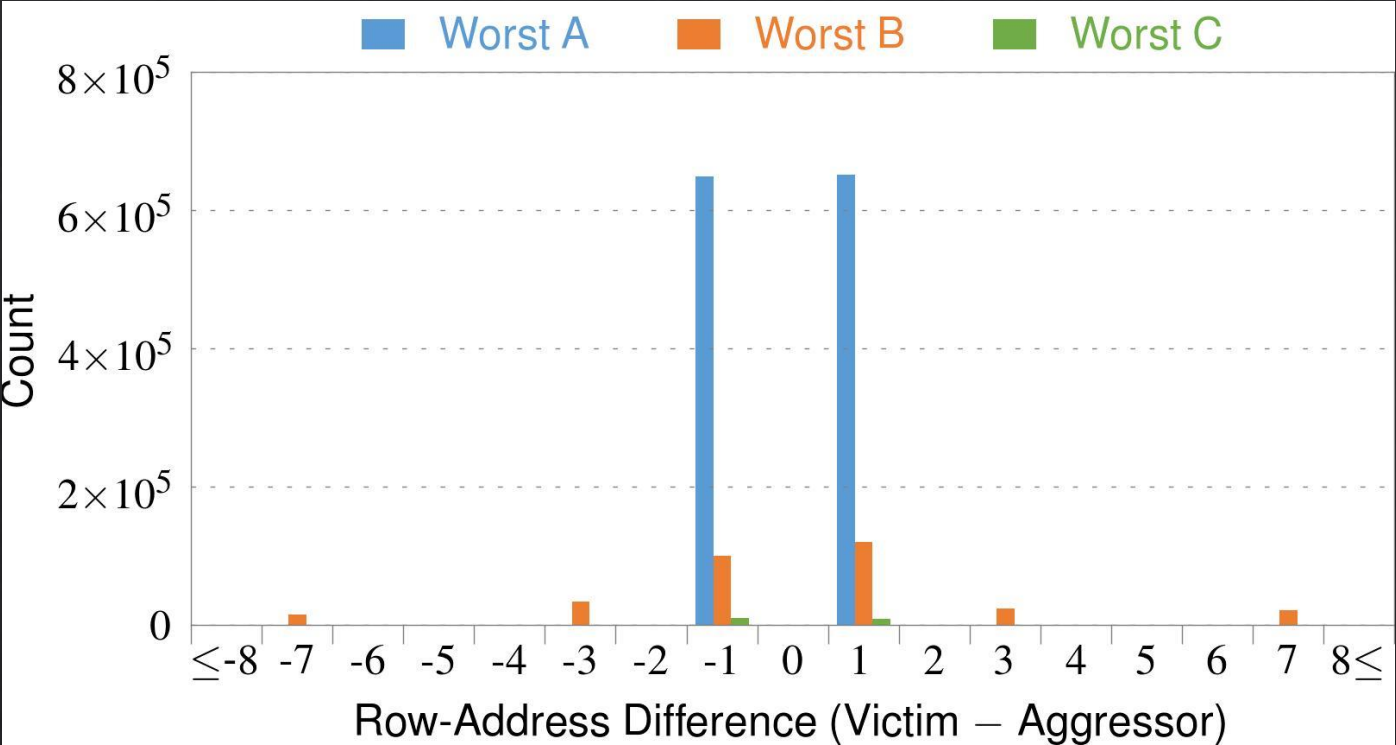
§ We report DRAM die versions marked on the chip packages, which typically progress in the following manner:  $\mathcal{M} \rightarrow \mathcal{A} \rightarrow \mathcal{B} \rightarrow \mathcal{C} \rightarrow \dots$ .

Table 3. Sample population of 129 DDR3 DRAM modules, categorized by manufacturer and sorted by manufacture date

Tested DRAM Modules from 2008-2014

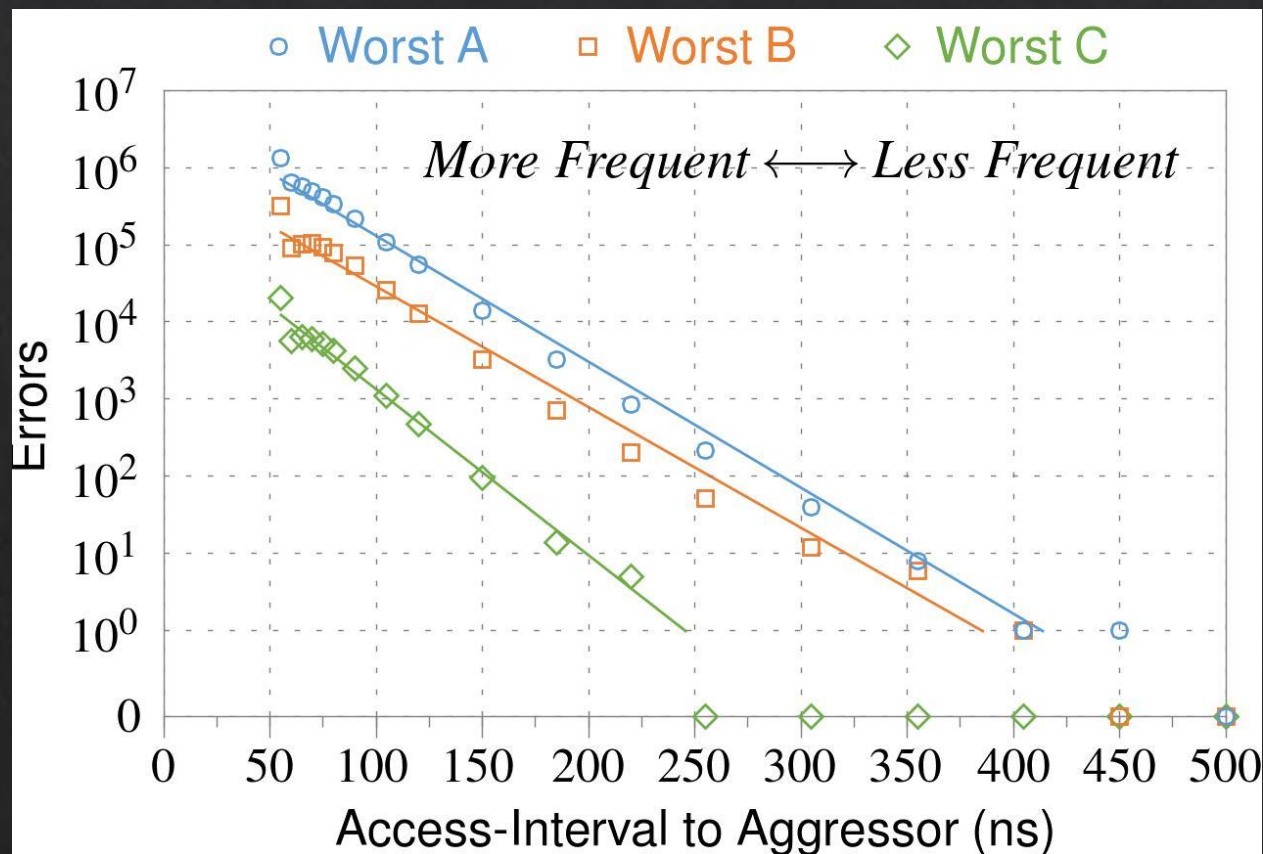
(129 total)

# Adjacency: Aggressor & Victim



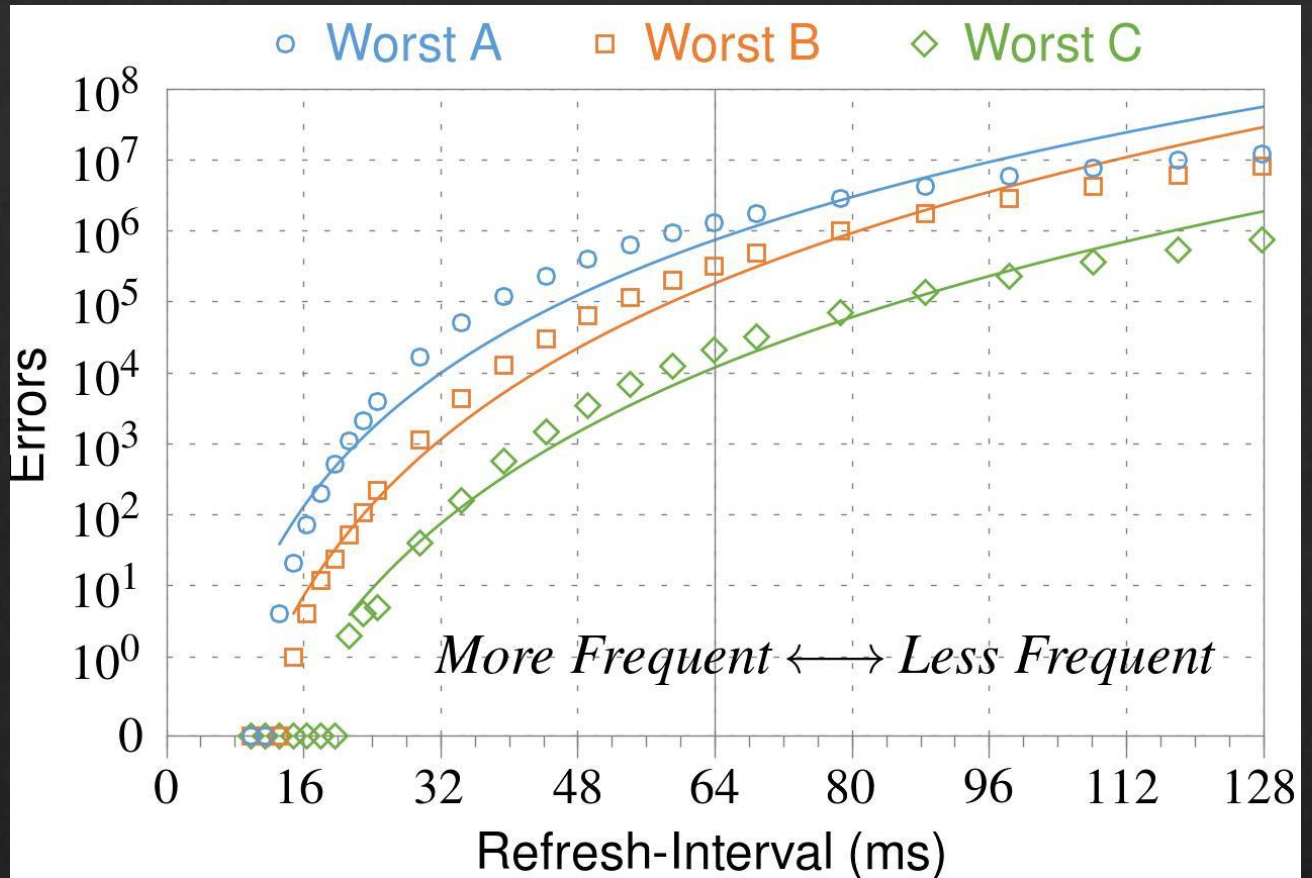
*Note: Using three modules with the most errors (only first bank)*

# 1 Access Interval (Aggressor)



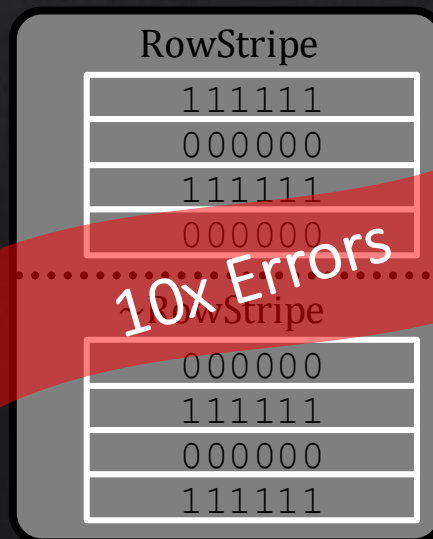
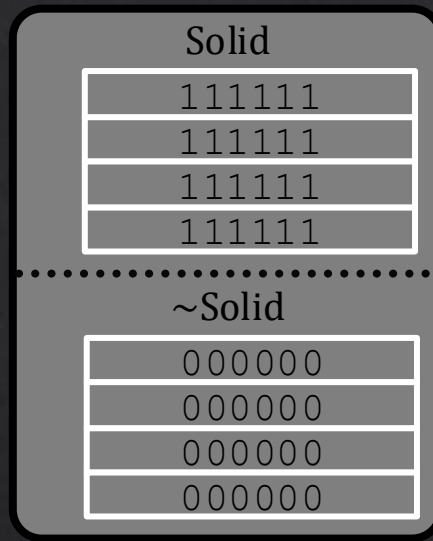
Note: Using three modules with the most errors (only first bank)

## 2 Refresh Interval



Note: Using three modules with the most errors (only first bank)

### 3 Data Pattern



# Other Key Observations

- ◆ *Victim Cells  $\neq$  Retention-Weak Cells*

- ◆ Almost no overlap between them

- ◆ *Errors are repeatable*

- ◆ Across ten iterations of testing, >70% of victim cells had errors in every iteration

- ◆ *As many as 4 errors per cache-line*

- ◆ Simple ECC (e.g., SECDED) cannot prevent all errors

- ◆ *Cells affected by two aggressors on either side*

- ◆ Double sided hammering

# RowHammer is Getting Much Worse

## Revisiting RowHammer: An Experimental Analysis of Modern DRAM Devices and Mitigation Techniques

Jeremie S. Kim<sup>§†</sup> Minesh Patel<sup>§</sup> A. Giray Yağlıkçı<sup>§</sup>  
Hasan Hassan<sup>§</sup> Roknoddin Azizi<sup>§</sup> Lois Orosa<sup>§</sup> Onur Mutlu<sup>§†</sup>  
<sup>§</sup>*ETH Zürich* <sup>†</sup>*Carnegie Mellon University*

(2020)

## Understanding RowHammer Under Reduced Wordline Voltage: An Experimental Study Using Real DRAM Devices

A. Giray Yağlıkçı<sup>1</sup> Haocong Luo<sup>1</sup> Geraldo F. de Oliveira<sup>1</sup> Ataberk Olgun<sup>1</sup> Minesh Patel<sup>1</sup>  
Jisung Park<sup>1</sup> Hasan Hassan<sup>1</sup> Jeremie S. Kim<sup>1</sup> Lois Orosa<sup>1,2</sup> Onur Mutlu<sup>1</sup>  
<sup>1</sup>*ETH Zürich* <sup>2</sup>*Galicia Supercomputing Center (CESGA)*

(2022)

## An Experimental Analysis of RowHammer in HBM2 DRAM Chips

Ataberk Olgun<sup>1</sup> Majd Osseiran<sup>1,2</sup> A. Giray Yağlıkçı<sup>1</sup> Yahya Can Tuğrul<sup>1</sup>  
Haocong Luo<sup>1</sup> Steve Rhyner<sup>1</sup> Behzad Salami<sup>1</sup> Juan Gomez Luna<sup>1</sup> Onur Mutlu<sup>1</sup>  
<sup>1</sup>*SAFARI Research Group, ETH Zürich* <sup>2</sup>*American University of Beirut*

(2023)

# Rowhammer simulation

## ◆ Demo

<https://jbhong.github.io/rowhammer/>



◆ Note, this is a simulator, not a real rowhammer emulator. To emulate, we need this:



# Rowhammer simulation

## Rowhammer Workbench

Built by [Jin Hong](#) for CITS3006 Penetration Testing, The University of Western Australia. Free to use and adapt for teaching, with credit.

Start here – how memory works

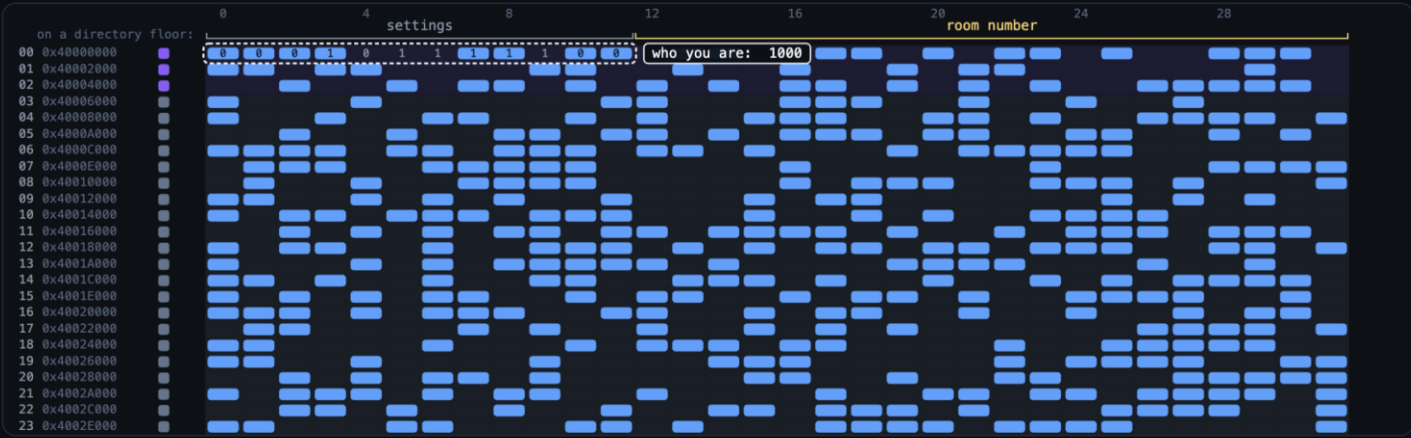
☒ Show the technical words

CHIP NUMBER 11

Try a different chip

Same number, same ch

Pedagogical model of the Rowhammer mechanism – not a circuit or cycle-accurate simulator. Weak cells are far denser than on a real chip, and charge is drawn on an expanded scale so leakage is visible at all. Each row shows one directory entry; a real 8 KB row holds about a thousand. Activation counts and timings are real.





# RowHammer Solutions

# Naive Solutions

## ◆ ① Throttle accesses to same row

- ◆ Limit access-interval:  $\geq 500\text{ns}$
- ◆ Limit number of accesses:  $\leq 128\text{K}$  ( $= 64\text{ms}/500\text{ns}$ )

## ◆ ② Refresh more frequently

- ◆ Shorten refresh-interval by  $\sim 7\times$

# Two Types of RowHammer Solutions

---

## ◆ Immediate

- ◆ To protect the vulnerable DRAM chips in the field
- ◆ Limited possibilities

## ◆ Longer-term

- ◆ To protect future DRAM chips
- ◆ Wider range of protection mechanisms

# Some Potential Solutions (ISCA 2014)

---

- Make better DRAM chips

Cost

- Refresh frequently

Power, Performance

- Sophisticated ECC

Cost, Power

- Access counters

Cost, Power, Complexity

# Apple's Security Patch for RowHammer

◇ <https://support.apple.com/en-gb/HT204934>

Available for: OS X Mountain Lion v10.8.5, OS X Mavericks v10.9.5

Impact: A malicious application may induce memory corruption to escalate privileges

Description: A disturbance error, also known as Rowhammer, exists with some DDR3 RAM that could have led to memory corruption. This issue was mitigated by increasing memory refresh rates.

CVE-ID

CVE-2015-3693 : Mark Seaborn and Thomas Dullien of Google, working from original research by Yoongu Kim et al (2014)

HP, Lenovo, and many other vendors released similar patches

# RowHammer Solution: PARA

## ◆ PARA: Probabilistic Adjacent Row Activation

### ◆ Key Idea

- ◆ After closing a row, we activate (i.e., refresh) one of its neighbors with a low probability:  $p = 0.005$

### ◆ Reliability Guarantee

- ◆ When  $p=0.005$ , errors in one year:  $9.4 \times 10^{-14}$
- ◆ By adjusting the value of  $p$ , we can vary the strength of protection against errors

# Advantages of PARA

## ◆ PARA refreshes rows infrequently

- ◆ Low power
- ◆ Low performance-overhead
  - ◆ Average slowdown: 0.20% (for 29 benchmarks)
  - ◆ Maximum slowdown: 0.75%

## ◆ PARA is stateless

- ◆ Low cost
- ◆ Low complexity

## ◆ PARA is an effective and low-overhead solution to prevent disturbance errors

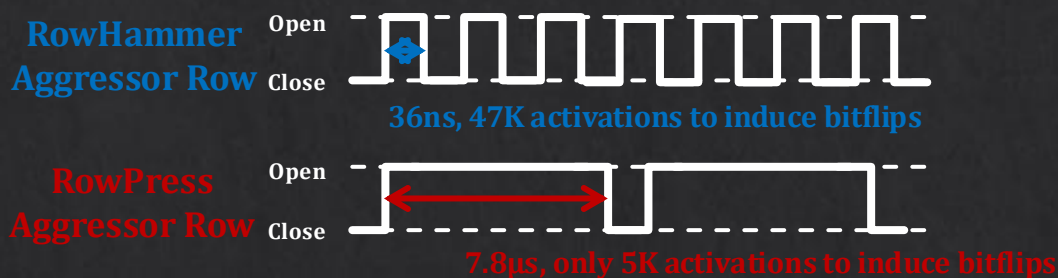


What next?

# RowPress vs. RowHammer

◆ Instead of using a high activation count,

🔧 increase the time that the aggressor row stays open



We observe bitflips even with **ONLY ONE activation** in extreme cases where the row stays open for 30ms

# Architecting Future Memory for Security

- ◆ Understand: Methods for vulnerability modeling & discovery
  - ◆ Modeling and prediction based on real (device) data and analysis
  - ◆ Understanding vulnerabilities
  - ◆ Developing reliable metrics
- ◆ Architect: Principled architectures with security as key concern
  - ◆ Good partitioning of duties across the stack
  - ◆ Cannot give up performance and efficiency
  - ◆ Patch-ability in the field
- ◆ Design & Test: Principled design, automation, (online) testing
  - ◆ Design for security
  - ◆ High coverage and good interaction with system reliability methods

# Two Major Future RowHammer Directions

## ◆ Understanding RowHammer

- ◆ Many effects still need to be rigorously examined
  - ◆ Aging of DRAM Chips
  - ◆ Environmental Conditions
  - ◆ Memory Access Patterns
  - ◆ Memory Controller & System Design Decisions
  - ◆ ...

## ◆ Solving RowHammer

- ◆ Flexible and Efficient RowHammer Solutions are necessary
  - ◆ In-field patchable / reconfigurable / programmable solutions
- ◆ Co-architecting System and Memory is important
  - ◆ To avoid performance and denial-of-service problems



# Google's Original RowHammer Attack

Presented by Mark Seaborn and Thomas Dullien at BlackHat 2015

# RowHammer Attack

## Kernel exploit

- x86 page tables entries (PTEs) are **dense and trusted**
  - They control access to physical memory
  - A bit flip in a PTE's physical page number can give a process access to a different physical page
- Aim of exploit: Get access to a page table
  - Gives access to all of physical memory
- Maximise chances that a bit flip is useful:
  - Spray physical memory with page tables
  - Check for useful, repeatable bit flip first

# RowHammer Attack

## x86-64 Page Table Entries (PTEs)

- Page table is a 4k page containing array of 512 PTEs
- Each PTE is 64 bits, containing:

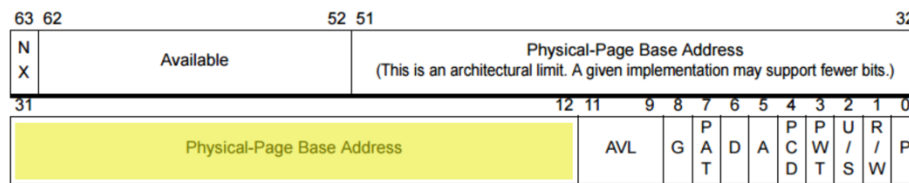
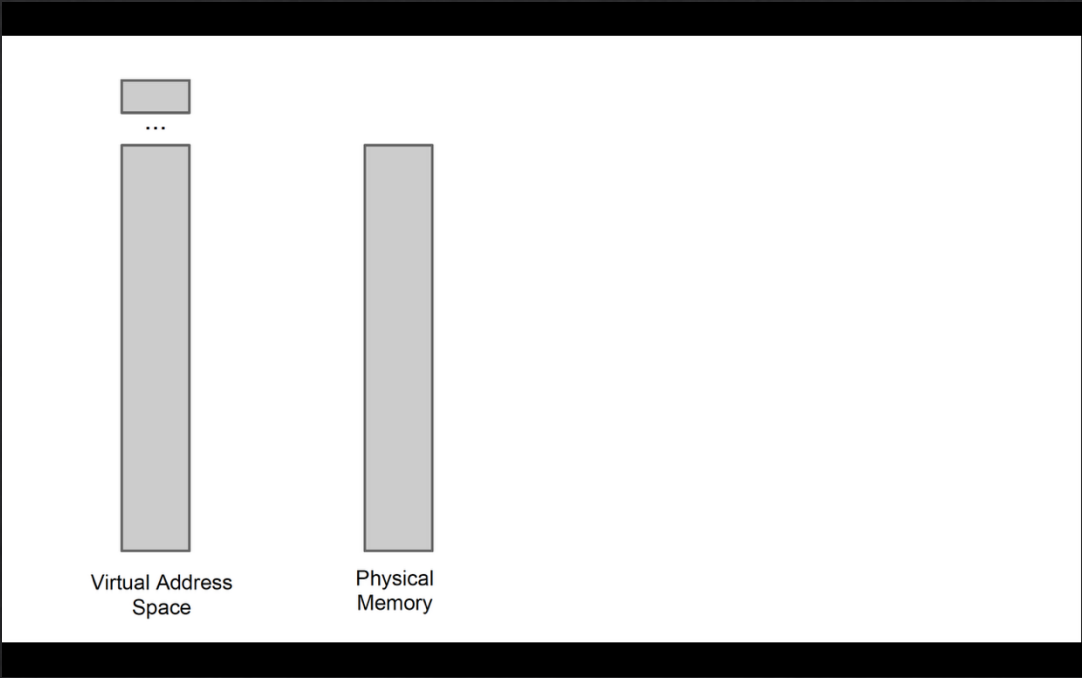
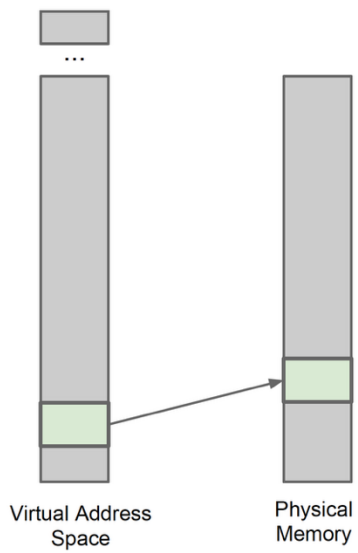


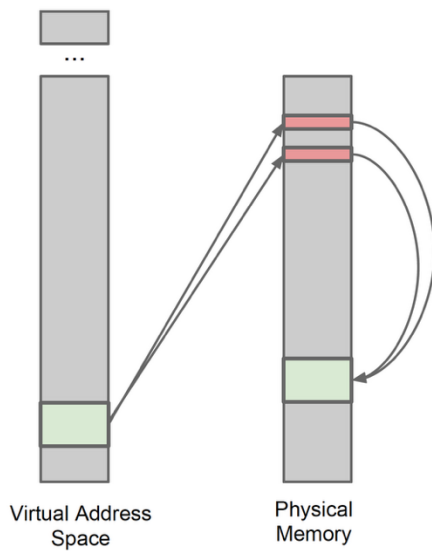
Figure 5-21. 4-Kbyte PTE—Long Mode

- Could flip:
  - “Writable” permission bit (RW): 1 bit → 2% chance
  - Physical page number: 20 bits on 4GB system → 31% chance

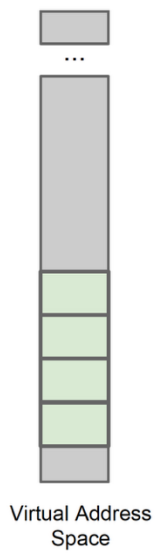




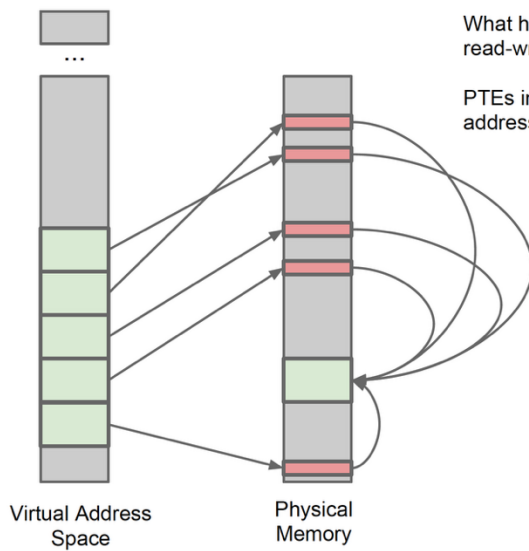
What happens when we map a file with read-write permissions?



What happens when we map a file with read-write permissions? Indirection via page tables.

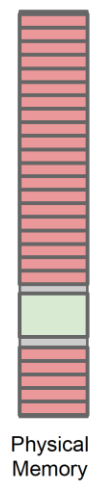
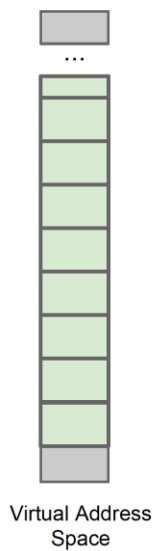


What happens when we repeatedly map a file with read-write permissions?



What happens when we repeatedly map a file with read-write permissions?

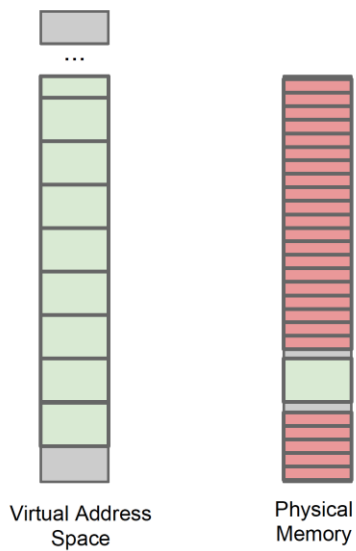
PTEs in physical memory help resolve virtual addresses to physical pages.



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

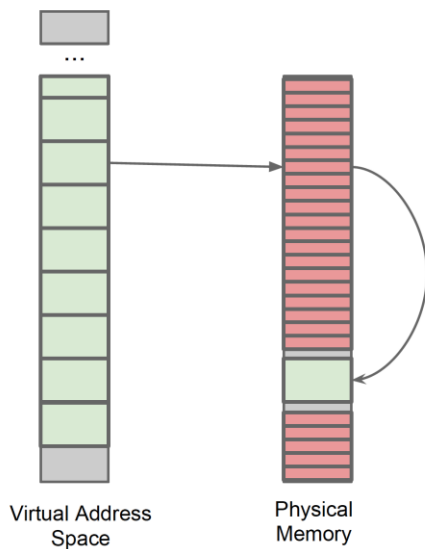


What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.



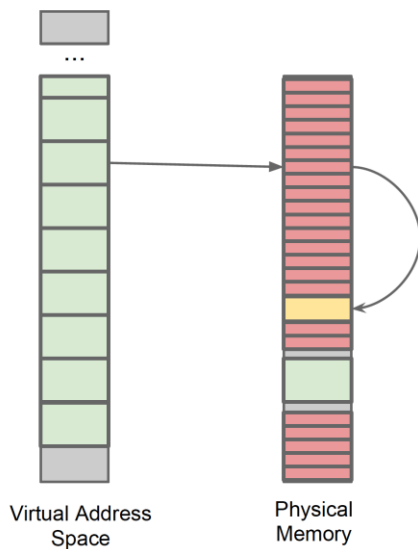
What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...



What happens when we repeatedly map a file with read-write permissions?

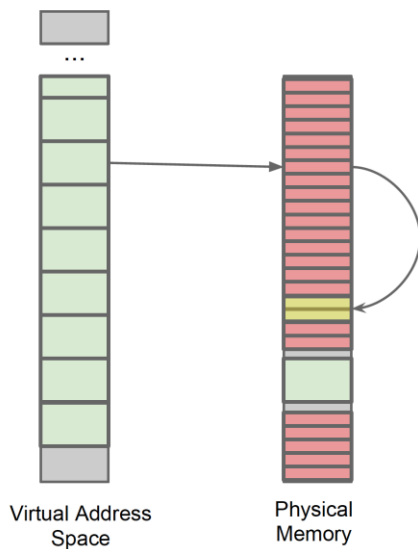
PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

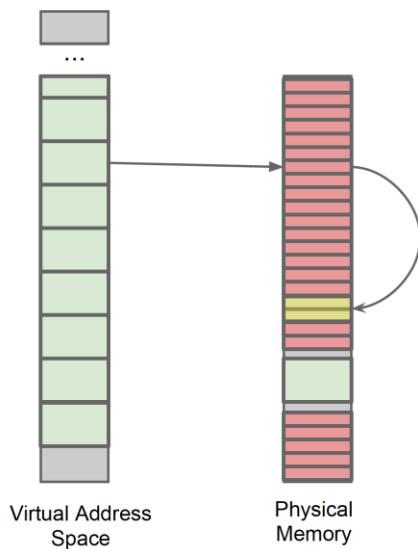
We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

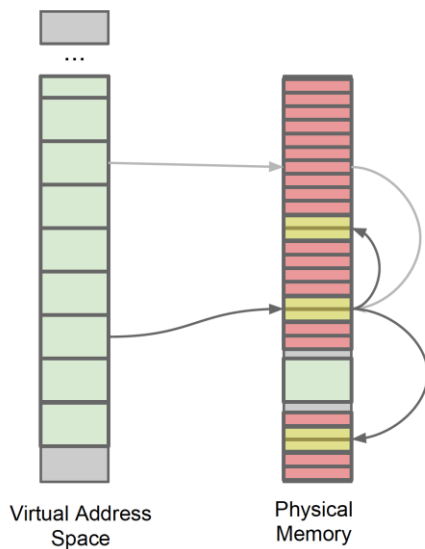
Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

An attacker that can read / write page tables ...



What happens when we repeatedly map a file with read-write permissions?

PTEs in physical memory help resolve virtual addresses to physical pages.

We can fill physical memory with PTEs.

Each of them points to pages in the same physical file mapping.

If a bit in the right place in the PTE flips ...

... the corresponding virtual address now points to a wrong physical page - with RW access.

Chances are this wrong page contains a page table itself.

An attacker that can read / write page tables can use that to map **any** memory read-write.

## Exploit strategy

Privilege escalation in 7 easy steps ...

1. Allocate a large chunk of memory
2. Search for locations prone to flipping
3. Check if they fall into the “right spot” in a PTE for allowing the exploit
4. Return that particular area of memory to the operating system
5. Force OS to re-use the memory for PTEs by allocating massive quantities of address space
6. Cause the bitflip - shift PTE to point into page table
7. Abuse R/W access to all of physical memory

In practice, there are many complications.

© 2013 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10986-1999

# More info

## ◆ Computer Architecture, Fall 2021, Lecture 5

- ◆ RowHammer (ETH Zürich, Fall 2021)
- ◆ <https://www.youtube.com/watch?v=7wVKnPj3NVw&list=PL5Q2soXY2Zi-Mnk1PxjEIG32HAGILkTOF&index=5>

## ◆ Computer Architecture, Fall 2021, Lecture 6

- ◆ RowHammer and Secure & Reliable Memory (ETH Zürich, Fall 2021)
- ◆ <https://www.youtube.com/watch?v=HNd4skQrt6I&list=PL5Q2soXY2Zi-Mnk1PxjEIG32HAGILkTOF&index=6>

## ◆ Rowhammer attack from Google

- ◆ <https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>
- ◆ <https://www.youtube.com/watch?v=0U7511Fb4to>

# References

---

## ◆ Materials adopted from

- ◆ Vyas Sekar – CMU
- ◆ Franziska Roesner – Washington U
- ◆ Onur Mutlu – ETH Zurich